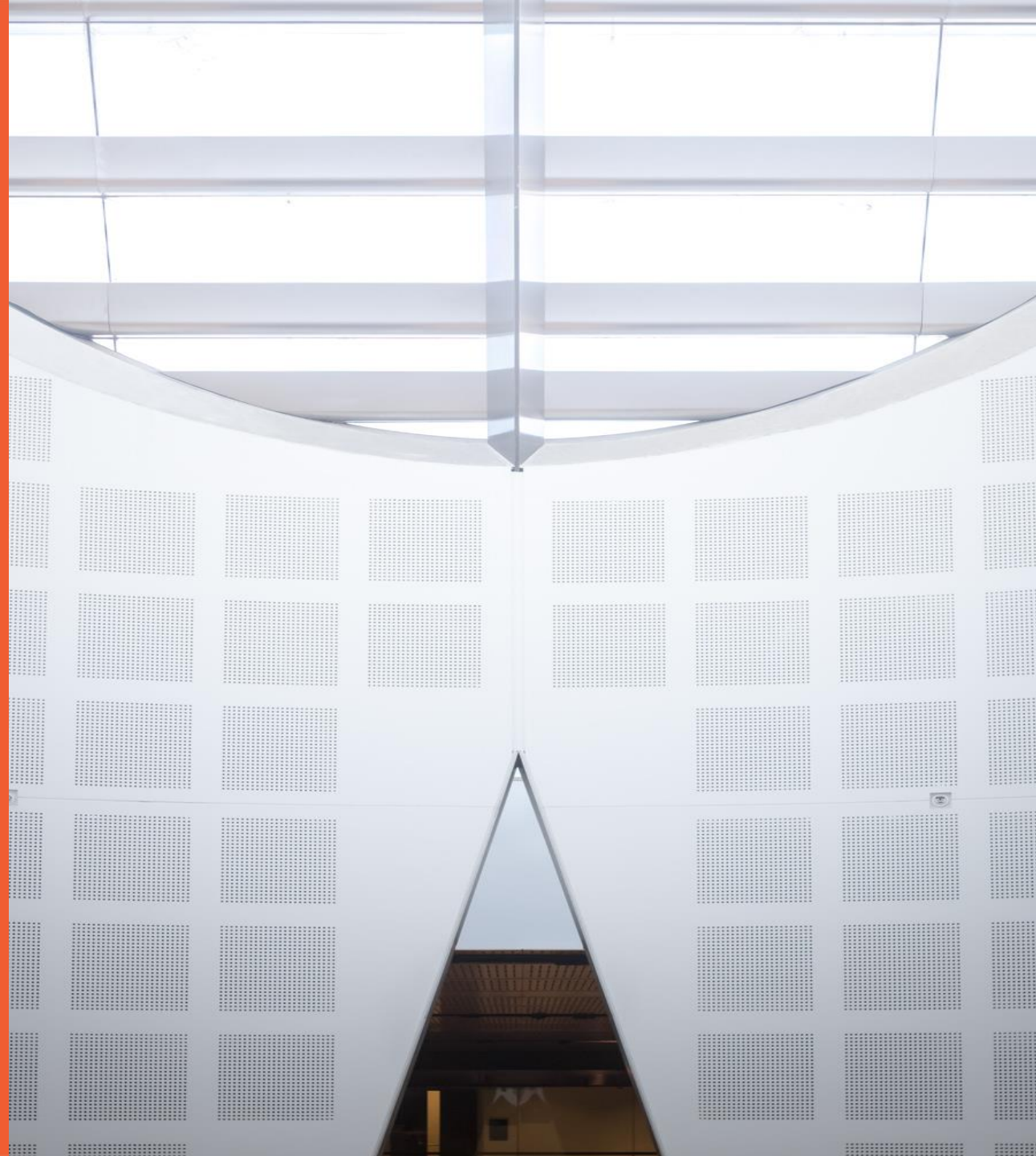
COMPx270: Randomised and Advanced Algorithms

Lecture 5: Graph algorithms

Clément Canonne

School of Computer Science

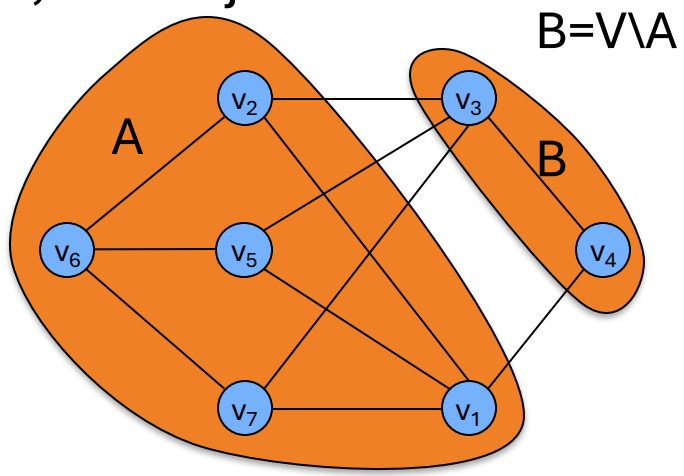
THE UNIVERSITY OF SYDNEY



Global Minimum Cut

Input: A connected, undirected graph G = (V, E).

For a set $A \subseteq V$ let $\delta(A) = \{(u,v) \in E : u \in A, v \in V \setminus A\}$.



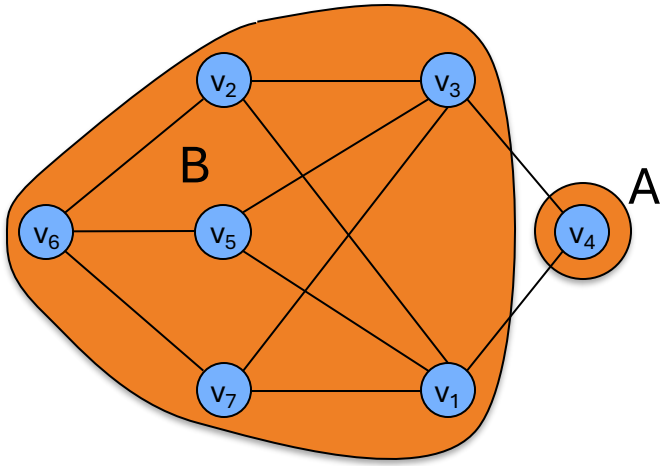
$|\delta(A)| = 4$

Aim: Find a cut (A, B) minimizing $|\delta(A)|$.

Global Minimum Cut

Input: A connected, undirected graph G = (V, E).

For a set $A \subseteq V$ let $\delta(A) = \{(u,v) \in E : u \in A, v \in V \setminus A\}$.



$|\delta(A)| = 2$

Aim: Find a cut (A, B) of minimum cardinality.

Global Minimum Cut

Applications: Partitioning items in a database, identifying clusters of related documents, network reliability, network design, circuit design, TSP solvers.

Network flow solution.

- Replace every edge (u, v) with two directed edges (u, v) and (v, u).
- Pick some vertex s and compute min s-v cut separating s from each other vertex $v \in V$.

Running time: $O((n-1) \cdot \text{MaxFlows})$

Global Minimum Cut

Running time: $O((n-1) \cdot \text{MaxFlows})$ — MaxFlows: ?

F: value of the max flow

Ford-Fulkerson:	$O(mF)$
*:	$O(m \log F)$
Edmonds-Karp:	$O(mn)$
Best (?):	$O(m \log \frac{n^2}{m})$

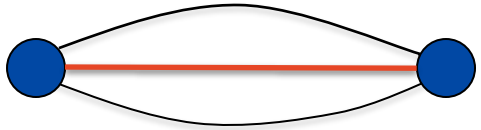
gives $O(n \cdot m \cdot \log \frac{n^2}{m})$ algo for min-cut

but $O(n^3)$ for dense graphs

All these are deterministic

Karger's Contraction Algorithm

Definition. A multigraph is a graph that allows multiple edges between a pair of vertices.



Karger's Contraction Algorithm

Definition. A multigraph is a graph that allows multiple edges between a pair of vertices.

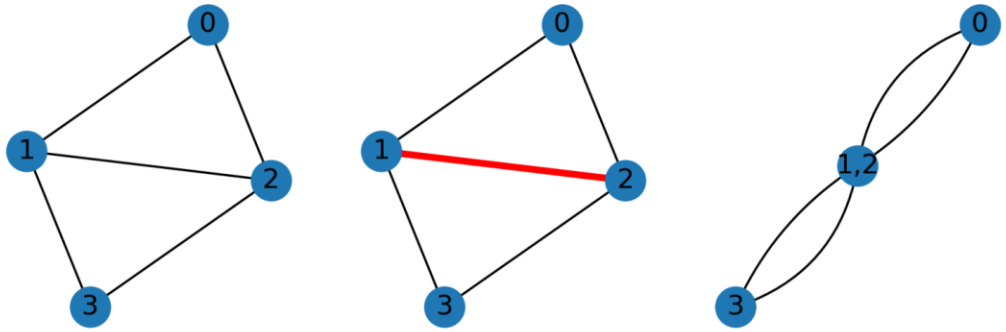
Karger's Contraction Algorithm

Let G=(V,E) be a multigraph (without self-loops).

The contraction of an edge e=(u,v)∈E gives G\e

- Replace u and v by single new super-node w
- Replace all edges (u,x) or (v,x) with an edge (w,x)
- Remove self-loops to w

} can be done in $O(n)$ time



Karger's Contraction Algorithm

(assume graph G is connected, o/w what are we doing here?)

Require: multigraph $G = (V, E)$

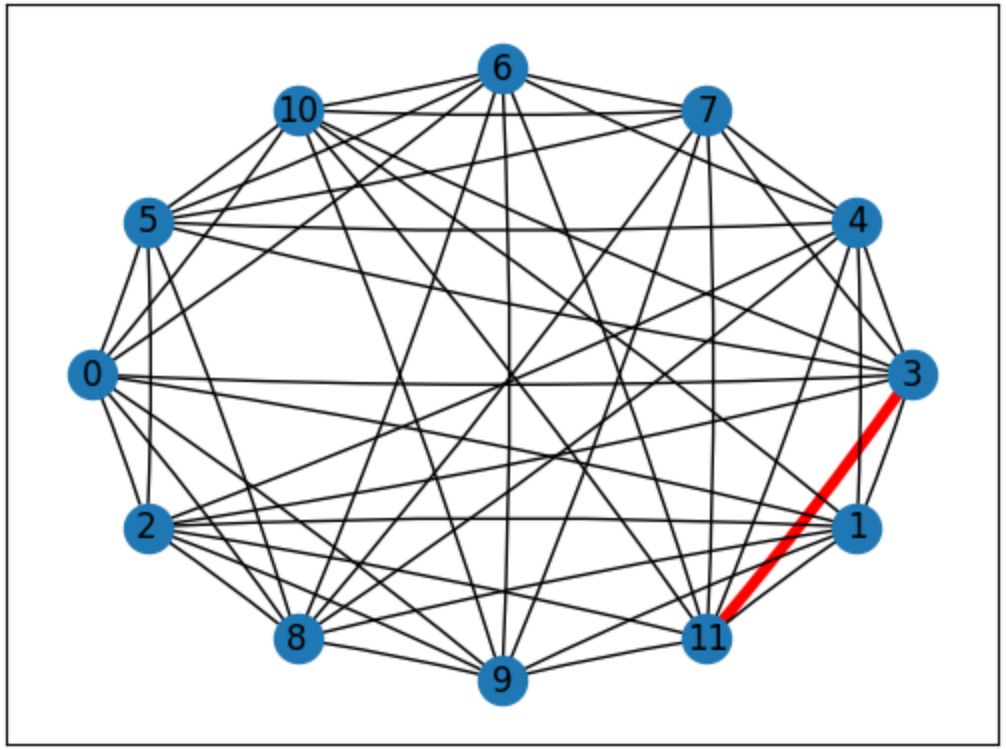
1: **while** $|V| > 2$ **do**
2: $O(n)$ Pick an edge $e \in E$ uniformly at random
3: $O(n)$ Contract it, and let $G \leftarrow G/e$
4: **return** the cut defined by the remaining two vertices.

$O(n^2)$ time

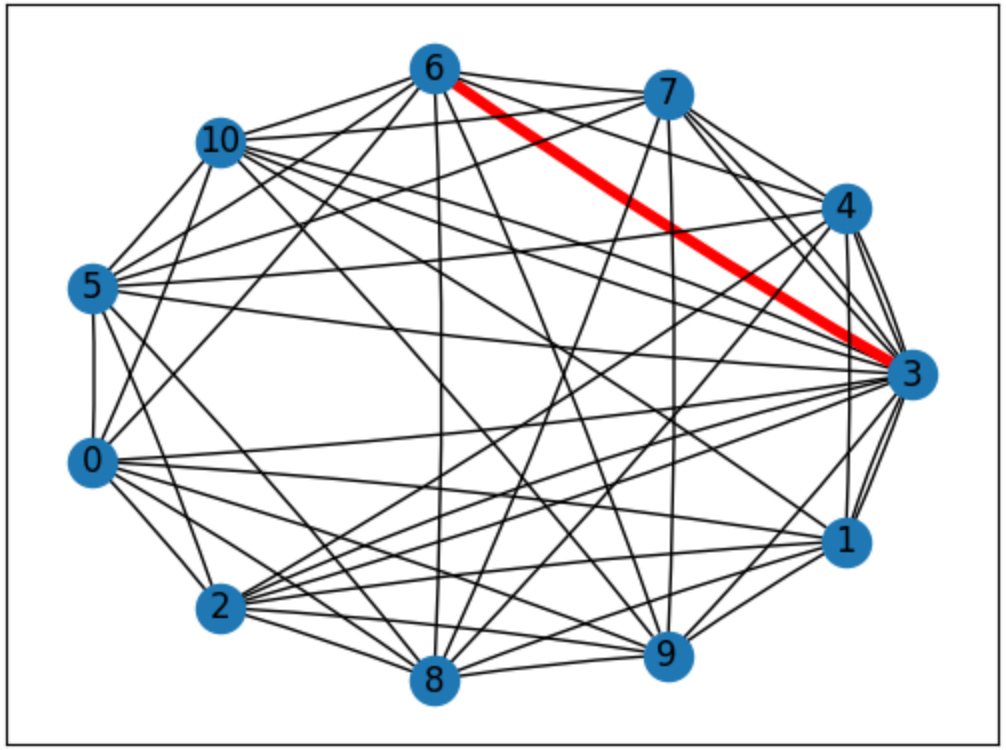
one fewer vertex

$n-2$ iterations

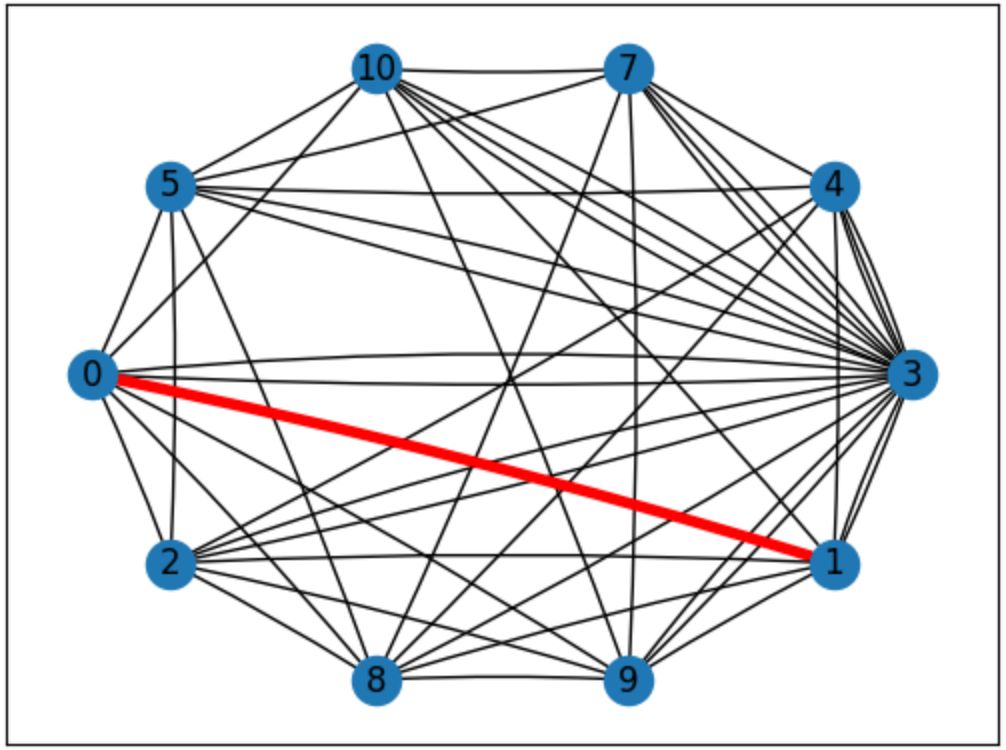
Karger's Contraction algorithm



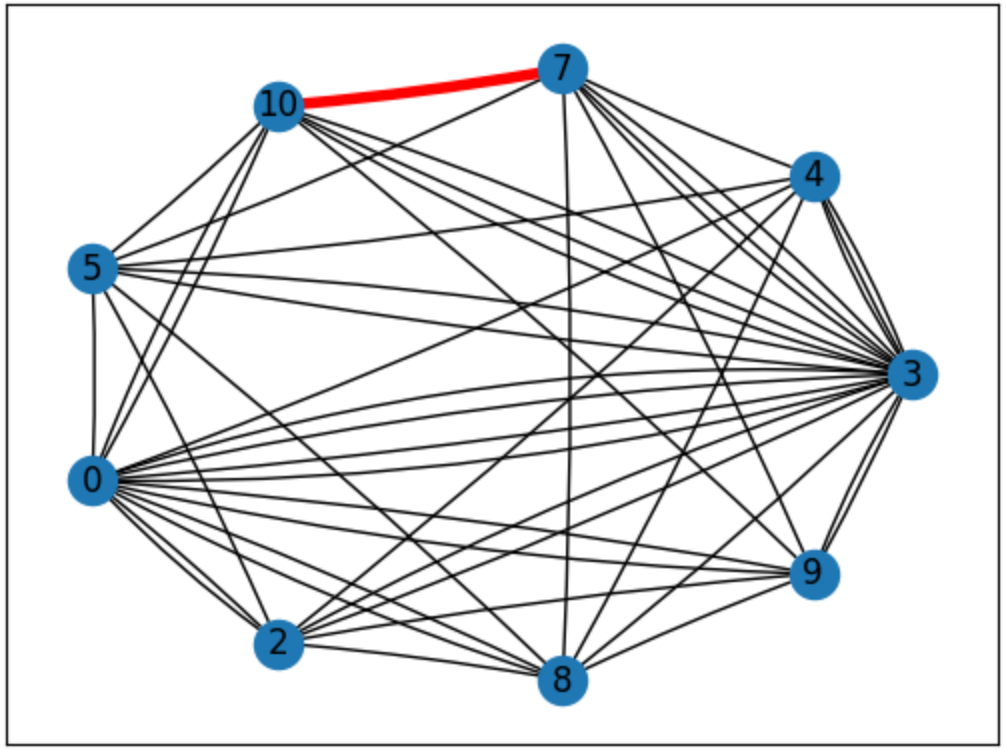
Karger's Contraction algorithm



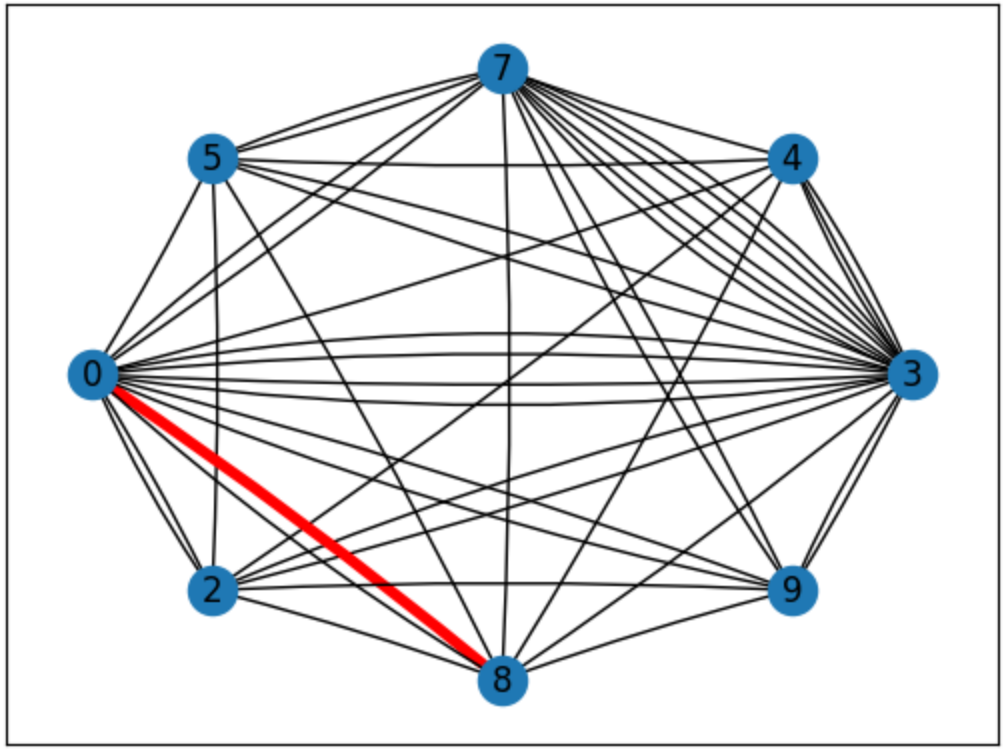
Karger's Contraction algorithm



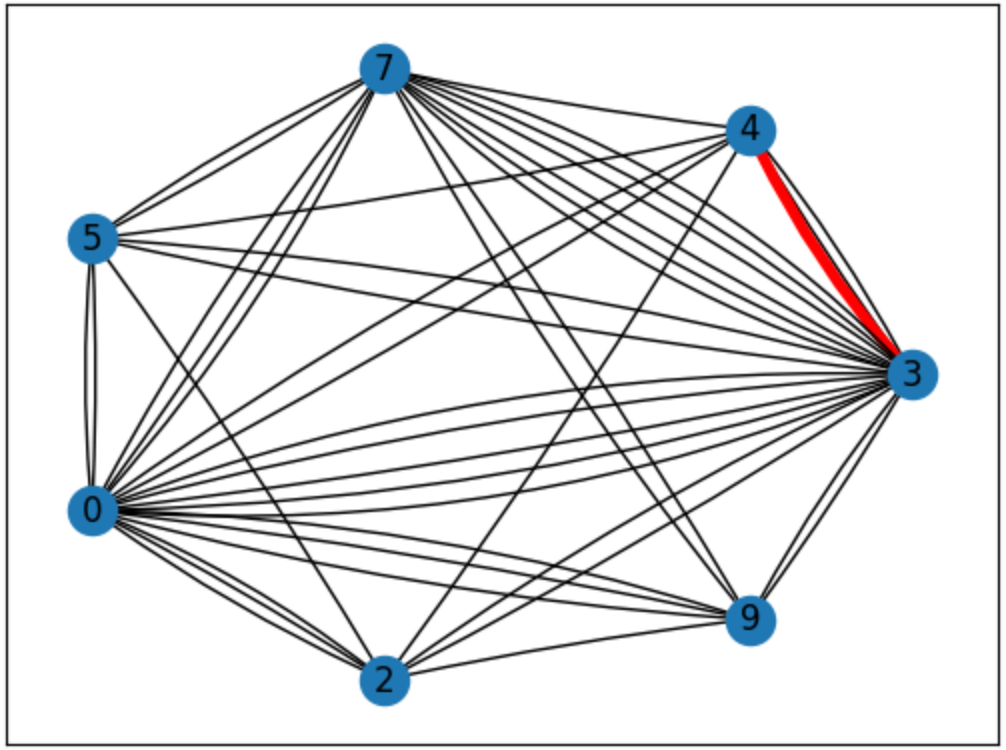
Karger's Contraction algorithm



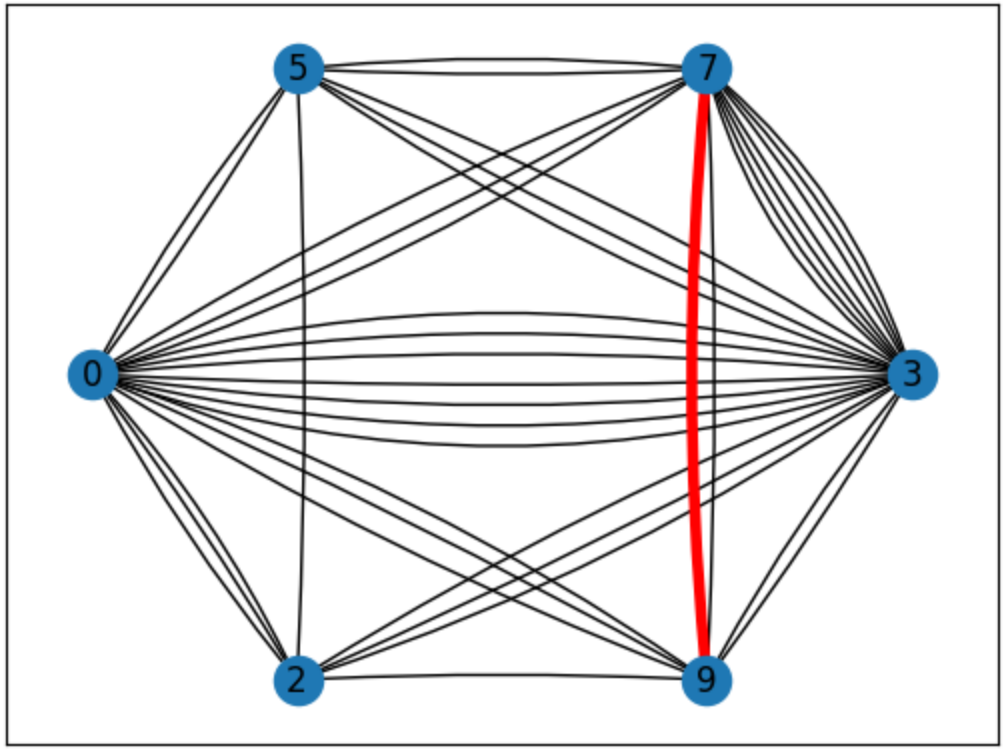
Karger's Contraction algorithm



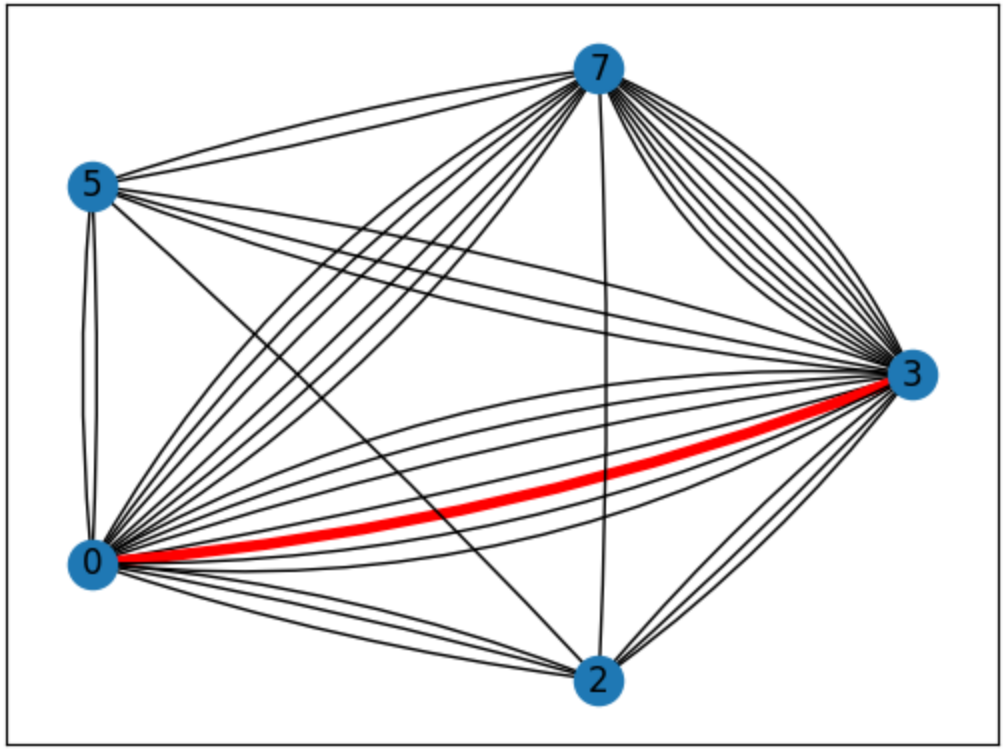
Karger's Contraction algorithm



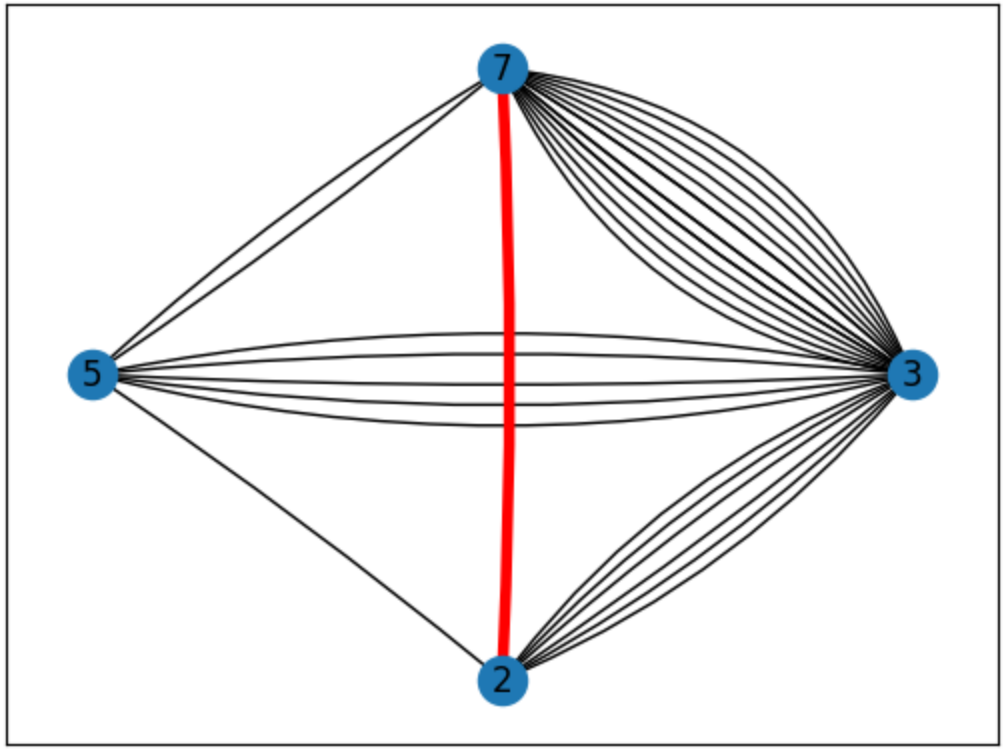
Karger's Contraction algorithm



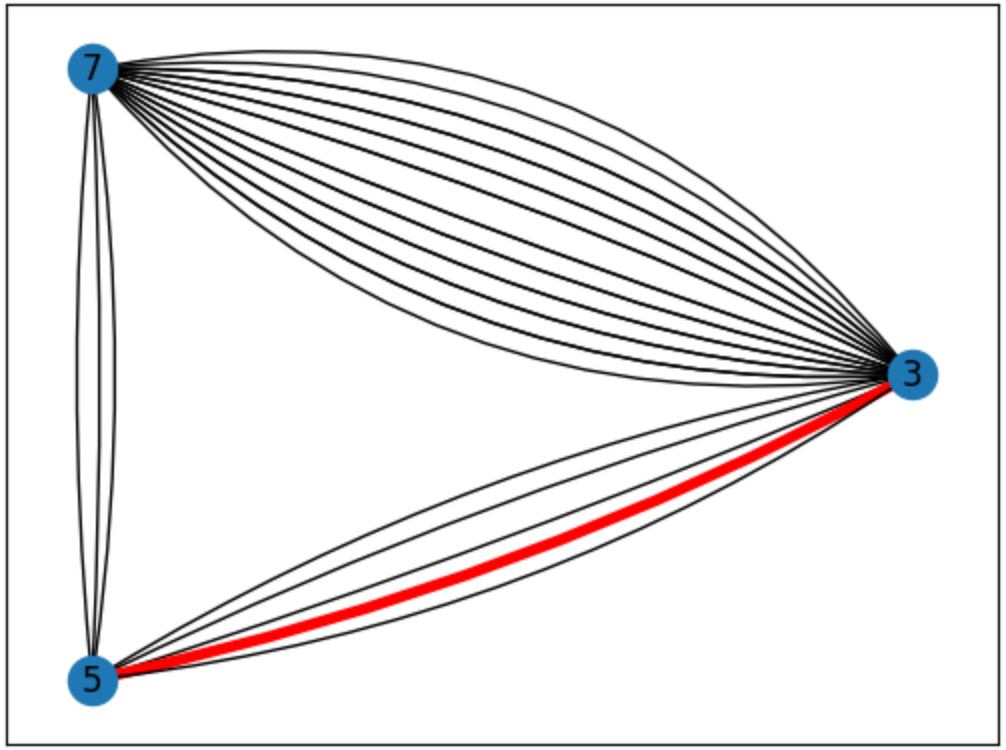
Karger's Contraction algorithm



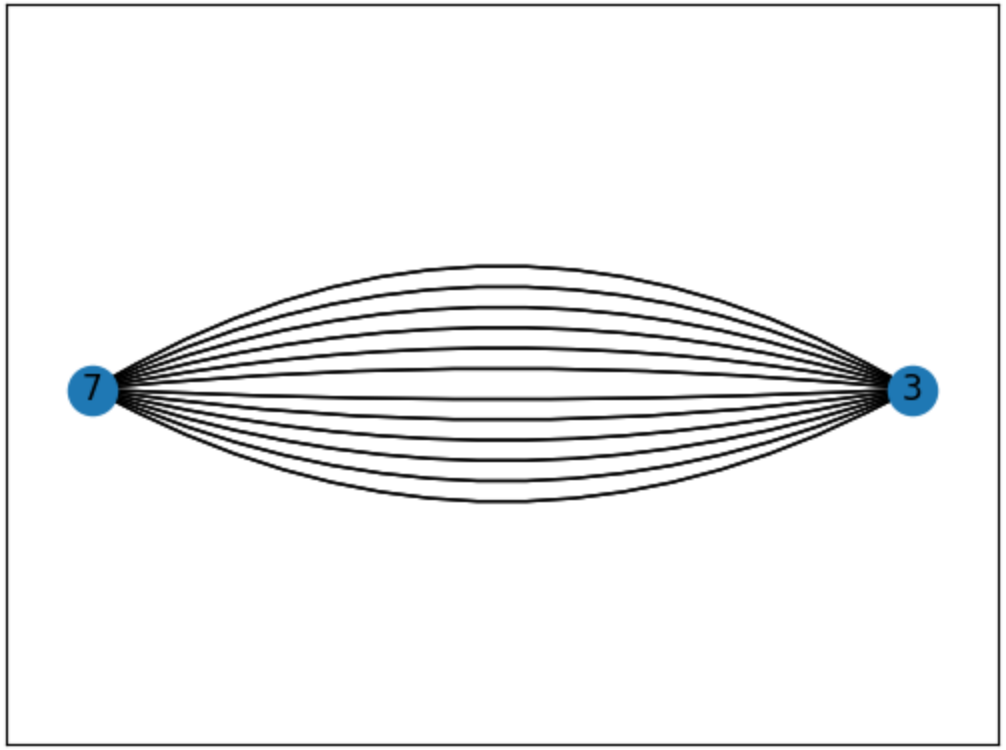
Karger's Contraction algorithm



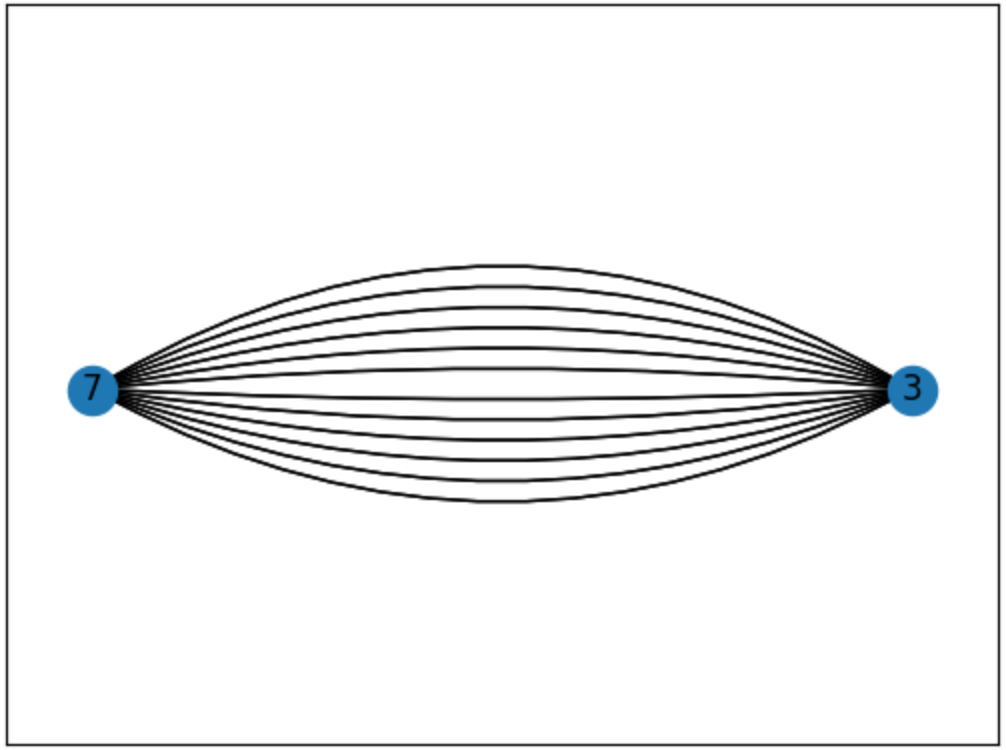
Karger's Contraction algorithm



Karger's Contraction algorithm



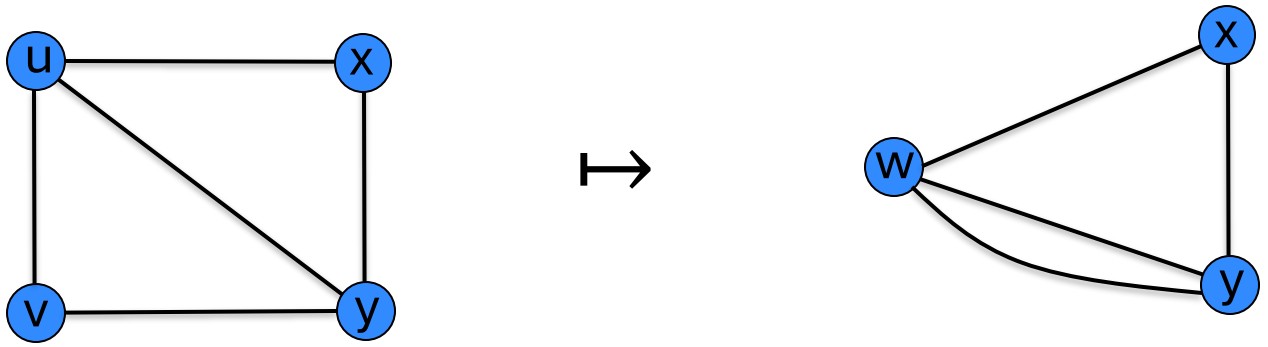
Karger's Contraction algorithm



The End.

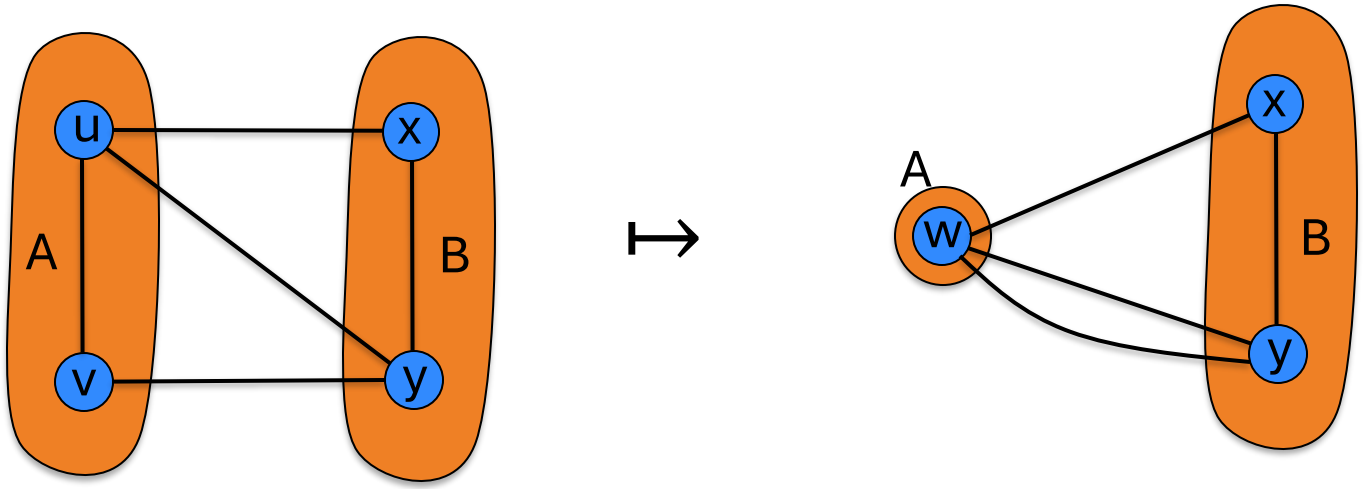
Karger's Contraction Algorithm

Observation: An edge (u,v) contraction preserves the cuts (A,B) where u and v are both in A or both in B.



Karger's Contraction Algorithm

Observation: An edge (u,v) contraction preserves the cuts (A,B) where u and v are both in A or both in B.



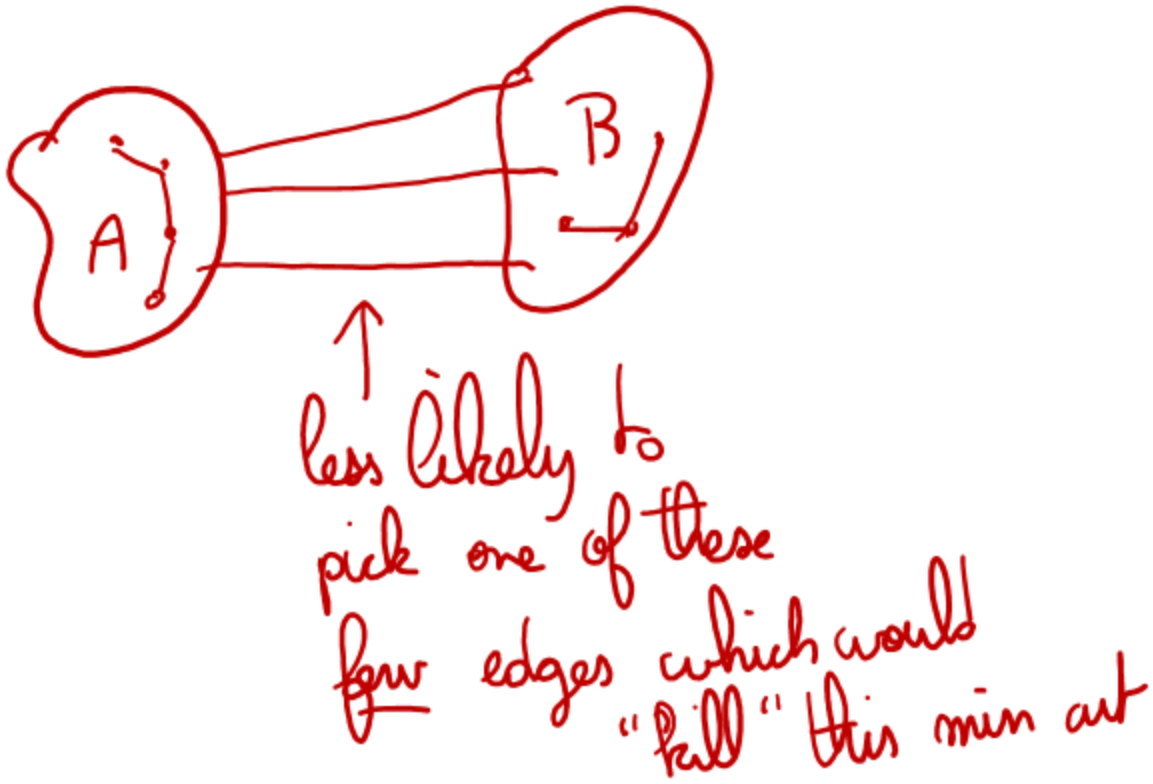
Karger's Contraction Algorithm

Observation: An edge (u,v) contraction preserves the cuts (A,B) where u and v are both in A or both in B.

If $u,v \in A$ then $\delta_G(A) = \delta_{G\setminus e}(A)$.
(with u and v replaced with "uv")

Karger's Contraction Algorithm

Observation: If (A,B) is a minimum cut, then we are less likely to choose an edge (u,v) crossing it!



Karger's Contraction Algorithm

Require: multigraph $G = (V, E)$

1: **while** $|V| > 2$ **do**

2: Pick an edge $e \in E$ uniformly at random

3: Contract it, and let $G \leftarrow G/e$

4: **return** the cut defined by the remaining two vertices.

Claim. This algorithm has a reasonable chance of finding a min cut.

↑

do not want exponentially small!

Key claim

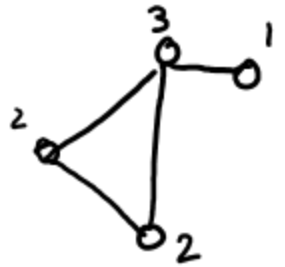
Claim. If C is a min-cut, then the algorithm returns it with probability at least $2/n^2$.

Useful

Handshaking lemma

Let $G=(V,E)$ be a (multi)graph.

Then $$|E| = \frac{1}{2}\sum_{v \in V} \deg v$$



$|E|=4$

$$\sum_{v \in V} \deg v = 2+2+3+1 = 8 = 2|E|$$

Key claim

Claim. If C is a min-cut, then the algorithm returns it with probability at least $2/n^2$.

Proof. Fix a min-cut C, let $k \triangleq |C|$. E_i = event that edge picked at iteration i did not "kill" C

$$\Pr[C \text{ is returned}] = \Pr[C \text{ is never killed}]$$
$$= \Pr[E_1 \cap E_2 \cap \cdots \cap E_{n-2}]$$
$$= \Pr[E_1] \cdot \Pr[E_2 \mid E_1] \cdot \Pr[E_3 \mid E_1 \cap E_2] \cdot \cdots \cdot \Pr[E_{n-2} \mid \bigcap_{i=1}^{n-3} E_i]$$

Let's look at one term:

$$\Pr[E_{i+1} \mid E_1 \cap \cdots \cap E_i]$$

After step i:
- C is still there (hasn't been killed)
- $G \setminus \{e_1, \ldots, e_i\} \triangleq G_i = (V_i, E_i)$
- $|V_i| = n - i$
- $|E_i| =$??

$$\Pr\Big[\underbrace{C \text{ is killed at step } i+1}_{E_{i+1}^c} \;\Big|\; \underbrace{C \text{ not killed so far}}_{\bigcap_{\ell \le i} E_\ell}\Big] = \frac{|C|}{|E_i|} = \frac{k}{|E_i|} \le \frac{2}{n-i}$$

$$|E_i| \underset{HSL}{=} \frac{1}{2}\sum_{v \in V_i} \deg v \ge \frac{1}{2}(n-i)k$$

↑ if $\exists\, v$ w/ $\deg v < k$ in E_i, corresponds to a cut in G (original graph) better than C!

and so

$$\Pr\Big[E_{i+1} \,\Big|\, \bigcap_{\ell \le i} E_i\Big] \overset{\text{A}}{\ge} 1 - \frac{2}{n-i}$$

$$\Pr[C \text{ survives}] = \prod_{i=0}^{n-3} \Pr[E_{i+1} \mid E_1 \cap \dots \cap E_i] \overset{\text{A}}{\ge} \prod_{i=0}^{n-3}\left(1 - \frac{2}{n-i}\right) = \prod_{i=0}^{n-3} \frac{n-i-2}{n-i}$$

$$= \prod_{j=3}^{n} \frac{j-2}{j} = \frac{1 \cdot 2 \cdot 3 \cdot 4 \cdot \dots \cdot (n-2)}{3 \cdot 4 \cdot \dots \cdot (n-2)(n-1)n} = \frac{2}{(n-1)n} = \frac{1}{\binom{n}{2}}$$

$$\ge \frac{2}{n^2} \qquad \square$$

Amplification

To amplify the probability of success, run the contraction algorithm many times.

Require: multigraph $G = (V, E)$, integer T

1: **for** $1 \leq t \leq T$ **do** ▷ Use fresh (independent) random bits for each

2: Run Algorithm on G, let C_t be the output

3: **return** the smallest cut among all cuts $C_1, \ldots, C_T$ obtained

Amplification

To amplify the probability of success, run the contraction algorithm many times.

Claim. If we repeat the contraction algorithm $r\binom{n}{2}$ times with independent random choices, the probability that all runs fail is at most $(1/e)^r$.

$$\Pr[\text{no min cut is returned in } r\tbinom{n}{2} \text{ runs}] \le \left(1 - \frac{1}{\binom{n}{2}}\right)^{r\binom{n}{2}} = \left(\left(1 - \frac{1}{\binom{n}{2}}\right)^{\binom{n}{2}}\right)^{r}$$

$$\le \left(\left(e^{-\frac{1}{\binom{n}{2}}}\right)^{\binom{n}{2}}\right)^{r} = e^{-r}$$

$1 - x \le e^{-x}$

$1 + x \le e^{x}$ ★

run $T = \underbrace{\Theta(n^2)}_{r = \Theta(1) \text{ in the thing above}}$ times to get success probability $1 - \frac{1}{100} = 99\%$

Karger's Contraction Algorithm

Require: multigraph $G = (V, E)$

1: **while** $|V| > 2$ **do**
2: Pick an edge $e \in E$ uniformly at random
3: Contract it, and let $G \leftarrow G/e$
4: **return** the cut defined by the remaining two vertices.

$n-2$ iterations

$O(n)$ time (carefully)

Running time?

$O(n^2)$ for success proba $\frac{2}{n^2}$

$O(Tn^2) = O(n^4)$ for success proba 99% :(

Karger's Contraction Algorithm

Require: multigraph $G = (V, E)$

1: **while** $|V| > 2$ **do**

2: Pick an edge $e \in E$ uniformly at random

3: Contract it, and let $G \leftarrow G/e$

4: **return** the cut defined by the remaining two vertices.

Running time?

The algorithm is iterated $O(n^2 \log(1/\delta))$ times... total running time $O(n^4 \log(1/\delta))$.

Karger's Contraction Algorithm

Can we do better?

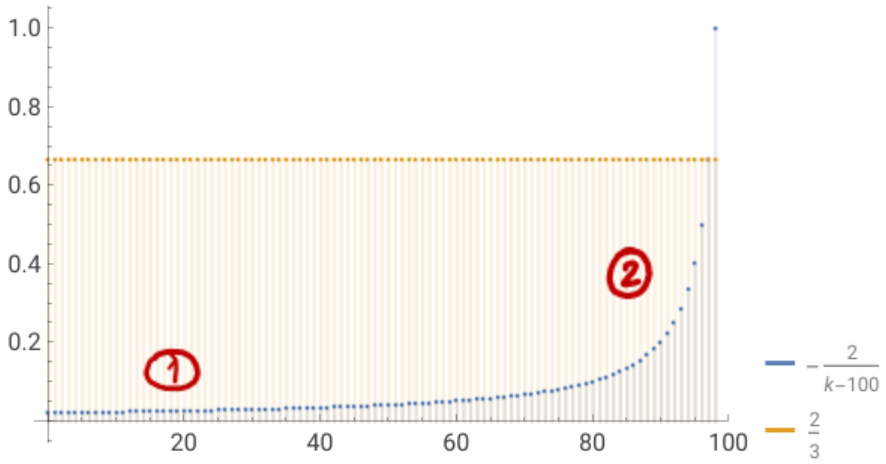
Karger's Contraction Algorithm

Can we do better?

Fix any min cut C.

$$\Pr[\,C \text{ is killed at step } i+1 \mid C \text{ survived until then}\,] \leq \frac{2}{n-i}$$

↑ kind of tight

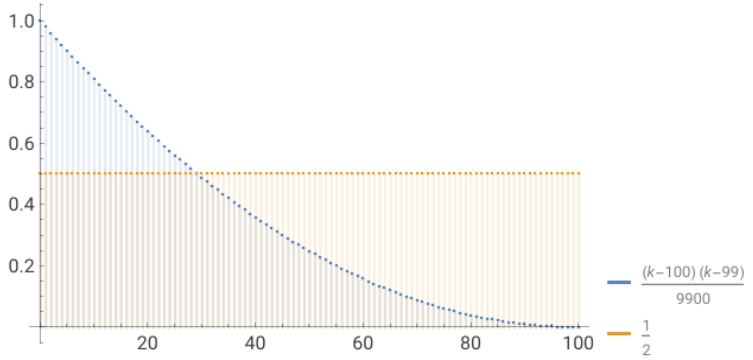


- At first, kill C
  - ① w/ very low proba : good
- At the end very likely to kill C : bad
  - ②

$$\Pr[\,C \text{ is killed after } n-s \text{ steps}\,] \leq \frac{s(s-1)}{n(n-1)} \approx \frac{s^2}{n^2}$$

Improved algorithm

Improvement. [Karger-Stein 1996]



- Early iterations are less risky than later ones: probability of contracting an edge in min cut hits 50% when $n/\sqrt{2}$ nodes remain.
- Run contraction algorithm until $n/\sqrt{2}$ nodes remain.
- Run contraction algorithm twice on resulting graph, and return best of two cuts.

Improved algorithm

```
1: procedure ModifiedKarger(G = (V, E), s)
2:     while |V| > s do
3:         Pick an edge e ∈ E uniformly at random
4:         Contract it, and let G ← G/e
5:     return G
6: procedure KargerStein(G = (V, E))
7:     if |V| ≤ 6 then
8:         return a minimum cut                ▷ Brute-force computation
9:     Set s ← ⌈n/√2 + 1⌉
10:    ▷ Contraction
11:        G_1 ← ModifiedKarger(G, s)
12:        G_2 ← ModifiedKarger(G, s)
13:    ▷ Recursion
14:        C_1 ← KargerStein(G_1)
15:        C_2 ← KargerStein(G_2)
16:    return the smallest cut among C_1, C_2
```

Improved algorithm: Karger-Stein

$O(|V|^2)$

```
1: procedure ModifiedKarger(G = (V, E), s)
2:     while |V| > s do
3:         Pick an edge e ∈ E uniformly at random
4:         Contract it, and let G ← G/e
5:     return G
6: procedure KargerStein(G = (V, E))
7:     if |V| ≤ 6 then
8:         return a minimum cut                ▷ Brute-force computation
9:     Set s ← ⌈n/√2 + 1⌉
10:    ▷ Contraction
11:        G1 ← ModifiedKarger(G, s)
12:        G2 ← ModifiedKarger(G, s)
13:    ▷ Recursion
14:        C1 ← KargerStein(G1)
15:        C2 ← KargerStein(G2)
16:    return the smallest cut among C1, C2
```

] $O(1)$

2 calls to $O(|V|^2)$

2 recursive calls

Running time?

$T(n) = 2T(\frac{n}{\sqrt{2}}) + O(n^2)$

recursive calls

modified Karger

$\rightsquigarrow T(n) = O(n^2 \log n)$

Improved algorithm: Karger-Stein

Running time?

```
1: procedure ModifiedKarger(G = (V, E), s)
2:     while |V| > s do
3:         Pick an edge e ∈ E uniformly at random
4:         Contract it, and let G ← G/e
5:     return G
6: procedure KargerStein(G = (V, E))
7:     if |V| ≤ 6 then
8:         return a minimum cut                ▷ Brute-force computation
9:     Set s ← ⌈n/√2 + 1⌉
10:    ▷ Contraction
11:        G_1 ← ModifiedKarger(G, s)
12:        G_2 ← ModifiedKarger(G, s)
13:    ▷ Recursion
14:        C_1 ← KargerStein(G_1)
15:        C_2 ← KargerStein(G_2)
16:    return the smallest cut among C_1, C_2
```

Improved algorithm: Karger-Stein

```
1: procedure ModifiedKarger(G = (V, E), s)
2:     while |V| > s do
3:         Pick an edge e ∈ E uniformly at random
4:         Contract it, and let G ← G/e
5:     return G
6: procedure KargerStein(G = (V, E))
7:     if |V| ≤ 6 then
8:         return a minimum cut                ▷ Brute-force computation
9:     Set s ← ⌈n/√2 + 1⌉
10:    ▷ Contraction
11:        G_1 ← ModifiedKarger(G, s)
12:        G_2 ← ModifiedKarger(G, s)
13:    ▷ Recursion
14:        C_1 ← KargerStein(G_1)
15:        C_2 ← KargerStein(G_2)
16:    return the smallest cut among C_1, C_2
```

Success probability?

Much better!

$\Omega\left(\frac{1}{\log n}\right)$

Improved algorithm: Karger-Stein

```
1: procedure ModifiedKarger(G = (V, E), s)
2:     while |V| > s do
3:         Pick an edge e ∈ E uniformly at random
4:         Contract it, and let G ← G/e
5:     return G
6: procedure KargerStein(G = (V, E))
7:     if |V| ≤ 6 then
8:         return a minimum cut                ▷ Brute-force computation
9:     Set s ← ⌈n/√2 + 1⌉
10:    ▷ Contraction
11:        G_1 ← ModifiedKarger(G, s)
12:        G_2 ← ModifiedKarger(G, s)
13:    ▷ Recursion
14:        C_1 ← KargerStein(G_1)
15:        C_2 ← KargerStein(G_2)
16:    return the smallest cut among C_1, C_2
```

Success probability?

$\underbrace{\Pr[\text{success}]}_{p(n)} = \Pr[C_1 \text{ or } C_2 \text{ is a min cut}]$

$= 1 - \Pr[C_1 \text{ not a min cut}] \cdot \Pr[C_2 \text{ not a min cut}]$

$p(n) \geq 1 - \left(1 - \frac{1}{2}p\left(\frac{n}{\sqrt{2}}\right)\right)\left(1 - \frac{1}{2}p\left(\frac{n}{\sqrt{2}}\right)\right) = 1 - \left(1 - \frac{1}{2}p\left(\frac{n}{\sqrt{2}}\right)\right)^2$

$= p\left(\frac{n}{\sqrt{2}}\right) - \frac{1}{4}p\left(\frac{n}{\sqrt{2}}\right)^2 \quad (\bigstar)$

Sketch: $f(t) = p(\sqrt{2}^t)$

(assuming $n = \sqrt{2}^t$)

$(\bigstar)$ becomes

$f(t) \geq f(t-1) - \frac{1}{4}f(t-1)^2$ "solves to" $p(n) = \Omega\left(\frac{1}{\log n}\right)$

Improved algorithm: Karger-Stein

Theorem. [Karger-Stein 1996] The Karger-Stein algorithm runs in time $O(n^2 \log n)$ and returns a min cut with probability at least $\Omega(1/\log n)$.

Corollary. The "best-of-T" Karger-Stein algorithm runs in time $O(n^2 \log^2 n)$ and returns a min cut with probability at least 99%.

Improved algorithm: Karger-Stein

Theorem. [Karger-Stein 1996] The Karger-Stein algorithm runs in time $O(n^2 \log n)$ and returns a min cut with probability at least $\Omega(1/\log n)$.

Corollary. The "best-of-T" Karger-Stein algorithm runs in time $O(n^2 \log^2 n)$ and returns a min cut with probability at least 99%

Best known. [Karger 2000] $O(m \log^3 n)$.

And now, for something completely different

And now, for something completely different?

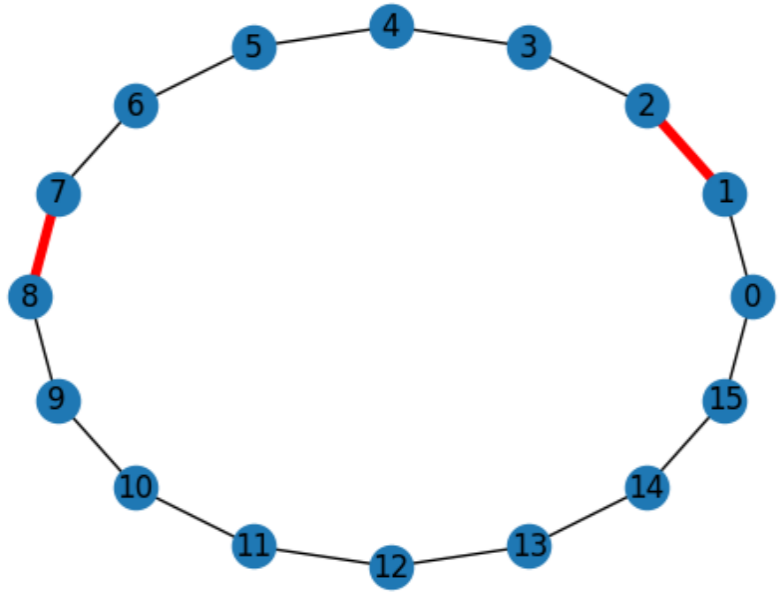
Theorem. An undirected graph G=(V,E) has at most _____ distinct min cuts.

And now, for something completely different?

Theorem. An undirected graph G=(V,E) has at most n(n-1)/2 distinct min cuts.

And now, for something completely different?

Theorem. An undirected graph G=(V,E) has at most n(n-1)/2 distinct min cuts. And this is tight.



And now, for something completely different?

Theorem. An undirected graph G=(V,E) has at most n(n-1)/2 distinct min cuts.

Proof. Karger's algo returns any fixed min cut C w/proba $\geq \frac{1}{\binom{n}{2}}$.

If there are M distinct min-cuts, it'll return one of them w/proba $\geq \frac{M}{\binom{n}{2}}$

$$1 \geq \Pr[\text{Karger's algo succeeds}] \geq \frac{M}{\binom{n}{2}}.$$

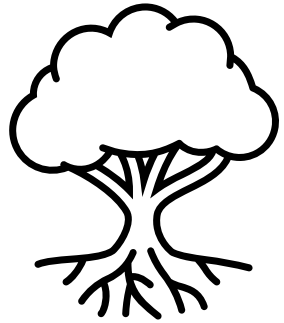
so $M \leq \binom{n}{2}$. □

And now, for something completely different?

How mind-blowing is this? Analyzing an algorithm proved a seemingly unrelated mathematical statement. 🤯

Compare this with last time, and the probabilistic method ("proof without algorithm!").

And now, for something else completely different!



Minimum Spanning Tree in (Expected) Linear Time: Karger-Klein-Tarjan.

MST via KKT: expected $O(m)$ time. *

Don't know any $O(m)$ time det. algo!

Kruskal: $O(m \log n)$
Bum's: $O(m + n \log n)$
Chazelle: $O(m \cdot \alpha(n))$
↑ some very slow-growing function

* You don't need to know the proof, but you need to know the statement and the key idea(s) of the algo] see lecture notes